

Deteccion de Senas Ecuatorianas — Guia de Integracion Movil

Este documento describe como llevar el modelo entrenado (YOLOv8n) a una aplicacion movil. Cubre la exportacion del modelo, los formatos recomendados por plataforma, y ejemplos de integracion en Android (Kotlin), iOS (Swift), Flutter y React Native. Tambien incluye consejos de rendimiento para ejecucion en dispositivos de gama baja.

1. Resumen del modelo

Arquitectura	YOLOv8n (nano)
Parametros	~3.2 M
Tamano .pt	~6 MB
Tamano TFLite FP16	~6 MB
Tamano TFLite INT8	~3 MB
Resolucion entrada	416x416 (recomendado movil); 320x320 (gama baja)
Clases (33)	10, 2, 4, 7, 8, 9, A, B, C, D, E, F, G, H, HOLA-5, I, K, L, M, MI, N, NOMBRE, O, P, Q, R, S, T, T
Inferencia movil	~30-60 FPS (Snapdragon 8xx / A14+); ~10-15 FPS (gama media)

2. Por que YOLOv8n para movil

- **Ligero:** ~6 MB y ~3.2 M parametros. Cabe sin problemas en cualquier APK/IPA.
- **Rapido:** inferencia en milisegundos, ideal para video en tiempo real.
- **Exportable:** ultralytics exporta nativamente a TFLite, ONNX, CoreML, NCNN.
- **Cuantizable:** INT8 reduce el tamano ~3x y acelera ~2-4x en CPUs ARM con perdida minima.
- **Detector + clasificador en uno:** da bounding box y clase de la sena al mismo tiempo.

3. Exportar el modelo

Una vez entrenado (best.pt en runs/detect/senas_v1/weights/), ejecuta:

```
# FP32 / FP16
python export_mobile.py

# INT8 cuantizado (mas chico y rapido en movil)
python export_mobile.py --int8 --imgsz 320
```

Esto produce, segun el sistema:

- **best.onnx** — multiplataforma, Android (NNAPI/onnxruntime-mobile), iOS, Flutter.
- **best_saved_model/best_float32.tflite** — Android y Flutter.
- **best_int8.tflite** — Android gama baja, Edge devices.
- **best.mlpackage** — iOS (solo se genera desde macOS).

4. Formato recomendado por plataforma

Plataforma	Formato recomendado	Runtime / SDK
Android nativo (Kotlin/Java)	TFLite (INT8)	TFLite Task Vision / LiteRT

iOS nativo (Swift)	CoreML (mlpackage)	Vision + CoreML
Flutter (Android + iOS)	TFLite	tflite_flutter / ultralytics_yolo
React Native	TFLite o ONNX	react-native-fast-tflite / onnxruntime-react-native
Cross-platform (alternativa)	ONNX	onnxruntime-mobile (Android + iOS)

5. Integracion en Android (Kotlin)

5.1. Dependencias en app/build.gradle:

```
dependencies {  
    implementation("org.tensorflow:tensorflow-lite:2.14.0")  
    implementation("org.tensorflow:tensorflow-lite-support:0.4.4")  
    implementation("org.tensorflow:tensorflow-lite-gpu:2.14.0")  
    implementation("androidx.camera:camera-camera2:1.3.1")  
    implementation("androidx.camera:camera-lifecycle:1.3.1")  
    implementation("androidx.camera:camera-view:1.3.1")  
}  
  
android {  
    aaptOptions { noCompress "tflite" }  
}
```

5.2. Colocar el modelo en `app/src/main/assets/senas.tflite` y un `labels.txt` con una clase por linea (mismo orden de data.yaml).

5.3. Cargar e inferir con CameraX + TFLite:

```
class YoloDetector(context: Context) {  
    private val interpreter: Interpreter  
    private val inputSize = 416  
    private val labels: List<String>  
  
    init {  
        val model = FileUtil.loadMappedFile(context, "senas.tflite")  
        val options = Interpreter.Options().apply { setNumThreads(4) }  
        interpreter = Interpreter(model, options)  
        labels = FileUtil.loadLabels(context, "labels.txt")  
    }  
  
    fun detect(bitmap: Bitmap, confTh: Float = 0.45f): List<Detection> {  
        val resized = Bitmap.createScaledBitmap(bitmap, inputSize, inputSize, true)  
        val input = TensorImage(DataType.FLOAT32).apply { load(resized) }  
        // Output YOLOv8: [1, 4 + nc, 8400]  
        val output = Array(1) { Array(4 + labels.size) { FloatArray(8400) } }  
        interpreter.run(input.buffer, output)  
        return postProcessYolov8(output[0], confTh, labels)  
    }  
}
```

5.4. Post-procesado (resumen):

- Transponer la matriz de salida y, por cada una de las 8400 anclas, obtener la clase con mayor probabilidad.
- Filtrar por umbral de confianza (ej. 0.45).
- Aplicar Non-Max Suppression con IoU ~0.5 para eliminar cajas duplicadas.
- Reescalar las coordenadas (x, y, w, h) de espacio 416x416 al tamaño de la imagen original.

Alternativa mas simple: usar la libreria **ultralytics-android** (MIT), que ya incluye preprocesado, postprocesado y NMS.

6. Integración en iOS (Swift)

6.1. Arrastra *best.mlpackage* dentro del proyecto Xcode. Xcode genera automáticamente una clase Swift con el nombre del modelo.

6.2. Inferencia con Vision (ya incluye NMS):

```
import Vision
import CoreML

class SignDetector {
    private let request: VNCoreMLRequest

    init() throws {
        let config = MLModelConfiguration()
        config.computeUnits = .all // CPU + GPU + Neural Engine
        let model = try VNCoreMLModel(for: best(configuration: config).model)
        request = VNCoreMLRequest(model: model)
        request.imageCropAndScaleOption = .scaleFill
    }

    func detect(pixelBuffer: CVPixelBuffer,
                completion: @escaping ([VNRecognizedObjectObservation]) -> Void) {
        let handler = VNImageRequestHandler(cvPixelBuffer: pixelBuffer,
                                             orientation: .right)

        try? handler.perform([request])
        let results = request.results as? [VNRecognizedObjectObservation] ?? []
        completion(results)
    }
}
```

6.3. Conectar con *AVCaptureSession* para video en tiempo real:

- Configura *AVCaptureVideoDataOutput* con formato **kCVPixelFormatType_32BGRA**.
- En el callback *captureOutput(_:didOutput:from:)*, pasa el *CVPixelBuffer* directo al detector — evita conversiones a *UIImage*.
- Dibuja las cajas sobre una capa *CAShapeLayer* sincronizada con el preview.

7. Integración en Flutter

La opción más directa es **ultralytics_yolo** (paquete oficial) que soporta cámara, detección y dibujado en una sola dependencia:

```
# pubspec.yaml
dependencies:
  ultralytics_yolo: ^0.1.21
  camera: ^0.10.5
// main.dart
final controller = UltralyticsYoloCameraController();
final predictor = ObjectDetector(
  modelPath: 'assets/senas.tflite',
  metadataPath: 'assets/metadata.yaml',
);
await predictor.loadModel(useGpu: true);

UltralyticsYoloCameraPreview(
  predictor: predictor,
  controller: controller,
  onCameraCreated: () => predictor.setConfidenceThreshold(0.45),
);
```

Si prefieres TFLite plano usa **tflite_flutter** + **tflite_flutter_helper** y replica el postprocesado YOLOv8 (la lógica es la misma que en Android).

8. Integracion en React Native

Opcion recomendada: **react-native-fast-tflite** (basado en JSI, ~zero copy).

```
npm i react-native-fast-tflite react-native-vision-camera
import { loadTensorflowModel } from 'react-native-fast-tflite';

const model = await loadTensorflowModel(require('./assets/senas.tflite'));
const output = model.runSync([inputTensor]);
// output[0] -> Float32Array de [1 * (4+nc) * 8400]
```

Combina con **react-native-vision-camera** y un frame processor para obtener el frame en formato adecuado para el modelo.

9. Consejos de rendimiento en movil

- **Reduce la resolucion** a 320x320 si necesitas mas FPS en gama baja.
- **Cuantiza a INT8** con datos representativos (data.yaml) — reduce el tamano ~3x.
- **Habilita aceleracion por hardware**: GPU delegate o NNAPI en Android, Neural Engine en iOS.
- **Procesa cada N frames** (ej. 1 de cada 2) y reutiliza el resultado anterior para los demas para mantener UI fluida.
- **Pre-asigna buffers** de entrada/salida; no los crees en cada frame.
- **Confianza minima alta** (0.5+) reduce falsos positivos y trabajo de NMS.
- **Imagen en RGB float32 / 255**: respeta el formato esperado por YOLOv8.

10. Flujo recomendado de extremo a extremo

- Entrenar: *python train.py* (50 epochs, imsz 416).
- Validar en webcam: *python webcam_test.py*.
- Exportar: *python export_mobile.py --int8*.
- Copiar *best_int8.tflite* a assets/ de la app + *labels.txt*.
- Integrar con CameraX (Android) / AVCaptureSession (iOS) / camera (Flutter).
- Probar en dispositivo real y medir FPS; ajustar imsz/cuantizacion si hace falta.

Generado automaticamente por generate_pdf.py